

Unit-4: Inheritance, Polymorphism and Wrapper classes

Main Topic	Subtopics	What You Learn
1. Inheritance	Concept of inheritance	Reusing properties of existing class
	Base class (superclass)	Parent class providing properties
	Derived class (subclass)	Child class inheriting features
	Types of inheritance	Single, Multilevel, Hierarchical, hybrid
	extends keyword	Used to inherit class
	super keyword	Access parent class members
2. Method Overriding	Runtime method overriding	Child class redefining parent method
	Rules of overriding	Same method signature
	Dynamic method dispatch	Method decided at runtime
3. Polymorphism	Concept of polymorphism	One interface, many forms
	Compile-time polymorphism	Method overloading
	Runtime polymorphism	Method overriding
	Dynamic binding	Method resolved during execution
4. Wrapper Classes	Wrapper class concept	Object representation of primitive types
	Primitive → object conversion	Autoboxing
	Object → primitive conversion	Unboxing
	Wrapper classes examples	Integer, Double, Character, Boolean

Unit-4: Inheritance in Java

Inheritance is one of the **most important concepts in Object-Oriented Programming (OOP)**.

It allows a new class to **reuse the properties and methods of an existing class**, which improves code reuse and maintainability.

1. Concept of Inheritance

Definition: Inheritance is the mechanism by which one class acquires the properties (variables) and behaviors (methods) of another class.

The existing class is called the **parent class**, and the new class is called the **child class**.

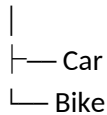
Purpose of Inheritance

Benefit	Explanation
Code reuse	Child class uses existing code
Reduces redundancy	Avoid rewriting same code
Improves maintainability	Changes needed only in parent class
Supports hierarchical design	Organizes classes logically

Simple Real-World Example

Vehicle System

Vehicle



Properties of Vehicle:

- speed
- fuel
- start()
- stop()

Car and Bike automatically get these properties.

Syntax of Inheritance

Java uses the **extends** keyword.

```
class ChildClass extends ParentClass {  
    // code  
}
```

Example Program

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats food");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();  
        d.bark();  
    }  
}
```

Output:

Animal eats food

Dog barks

The Dog class inherited the eat() method from Animal.

2. Base Class (Superclass)

Definition: The **base class (superclass)** is the class whose properties and methods are inherited by another class.

It is also called:

- parent class
- super class

Characteristics of Superclass

Feature	Description
Provides properties	Variables
Provides behaviors	Methods
Used as base for other classes	Derived classes inherit it

3. Derived Class (Subclass)

Definition: The **derived class (subclass)** is the class that inherits properties and methods from another class and it also have its own properties and methods.

It is also called:

- child class
- subclass

Example Showing Inheritance

```
class Person {
    String name;
    void displayName() {
        System.out.println(name);
    }
}
class Student extends Person {
    int roll;
    void displayRoll() {
        System.out.println(roll);
    }
}
class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Dhruv";
        s.roll = 101;
        s.displayName();
        s.displayRoll();
    }
}
```

Student inherited name and displayName() from Person.

Real-World Example

Employee System

Employee

|
├ name
├ salary
└ work()

Manager

|
├ department
└ manageTeam()

Manager inherits:

- name
- salary
- work()

Interface in Java:

An interface is used to achieve **100% abstraction** and define a **contract** that classes must follow.

1. What is an Interface?

Definition: An interface is a reference type in Java that contains **only abstract methods (by default)** and is declared using the interface keyword.

☞ It defines *what to do*, not *how to do it*

Syntax

```
interface InterfaceName {  
    void method1();  
    void method2();  
}
```

2. Key Features of Interface

Feature	Description
No object creation	Cannot create object
Methods	Abstract by default
Variables	public, static, final
Supports multiple inheritance	Yes
Implemented using	implements keyword

3. Example of Interface

```
interface Animal {  
    void sound();  
}
```

```
class Dog implements Animal {  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

Output

Dog barks

4. Important Rule ✂

- ☞ A class implementing interface must:
- Implement **all methods**
 - Use implements keyword

5. Multiple Inheritance Using Interface

☞ Java does not support multiple inheritance with classes

☞ But supports using interfaces

```
interface A {  
    void show();  
}
```

```
interface B {  
    void display();  
}
```

```
class Test implements A, B {  
    public void show() {  
        System.out.println("Show method");  
    }  
    public void display() {  
        System.out.println("Display method");  
    }  
}
```

```
}  
}
```

6. Interface Variables

☞ By default:

- public
- static
- final

```
interface Demo {  
    int x = 10; // public static final  
}
```

☞ Cannot change value:

```
x = 20; // ERROR ✕
```

7. Interface vs Abstract Class

Feature	Interface	Abstract Class
Methods	Only abstract (mostly)	Abstract + Normal
Variables	public static final	Any type
Constructor	Not allowed	Allowed
Inheritance	Multiple	Single
Keyword	interface	abstract

8. Real-World Example

Payment System

Interface	Implementation
Payment	CreditCard, UPI

```
interface Payment {  
    void pay();  
}  
  
class UPI implements Payment {  
    public void pay() {  
        System.out.println("Paid using UPI");  
    }  
}  
  
class Card implements Payment {  
    public void pay() {  
        System.out.println("Paid using Card");  
    }  
}
```

9. Default and Static Methods (Java 8+)

☞ Interface can have methods with body using default and static

```
interface Demo {  
    default void show() {  
        System.out.println("Default method");  
    }  
  
    static void display() {  
        System.out.println("Static method");  
    }  
}
```

10. Why Interface is Used

Purpose	Explanation
Abstraction	Hide implementation

Multiple inheritance	Achieved using interfaces
Flexibility	Loose coupling
Standardization	Define rules

11. Important Rules for Exam ✂

- Cannot create object of interface
- Use implements keyword
- All methods must be implemented
- Variables are always **final**
- Supports multiple inheritance

Unit-4: Types of Inheritance in Java

Inheritance allows a class to **acquire properties and methods from another class**.

Different class relationships form different **types of inheritance structures**.

Overview of Types of Inheritance

Type	Description
Single Inheritance	One child class inherits one parent class
Multilevel Inheritance	Inheritance chain (grandparent → parent → child)
Hierarchical Inheritance	Multiple child classes inherit one parent
Multiple Inheritance	One child inherits multiple parents
Hybrid Inheritance	Combination of multiple inheritance types

⚠ Important: **Java does not support multiple inheritance using classes** (to avoid ambiguity).

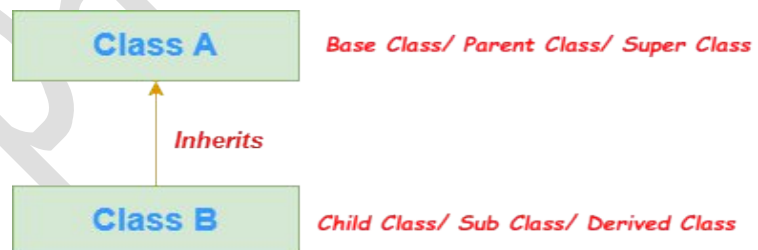
However, it can be achieved using **interfaces**.

1. Single Inheritance

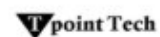
Concept: In **single inheritance**, one subclass inherits from one superclass.

Example

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```



Single Inheritance



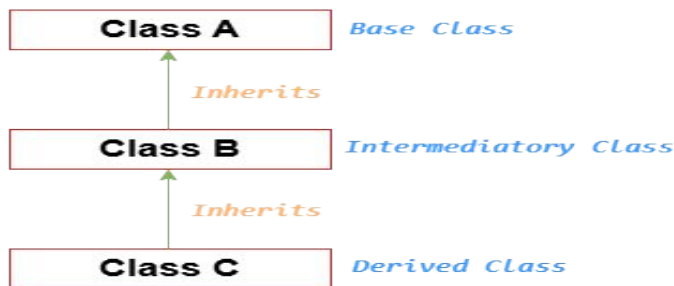
Output

Animal eats

Dog barks

2. Multilevel Inheritance

Concept : In **multilevel inheritance**, a class inherits from a class which itself inherits from another class.



Multilevel Inheritance



Example

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Puppy extends Dog {  
    void weep() {  
        System.out.println("Puppy weeps");  
    }  
}  
  
// Main program  
class Test {  
    public static void main(String[] args) {  
        Puppy p = new Puppy();  
        p.eat();  
        p.bark();  
        p.weep();  
    }  
}
```

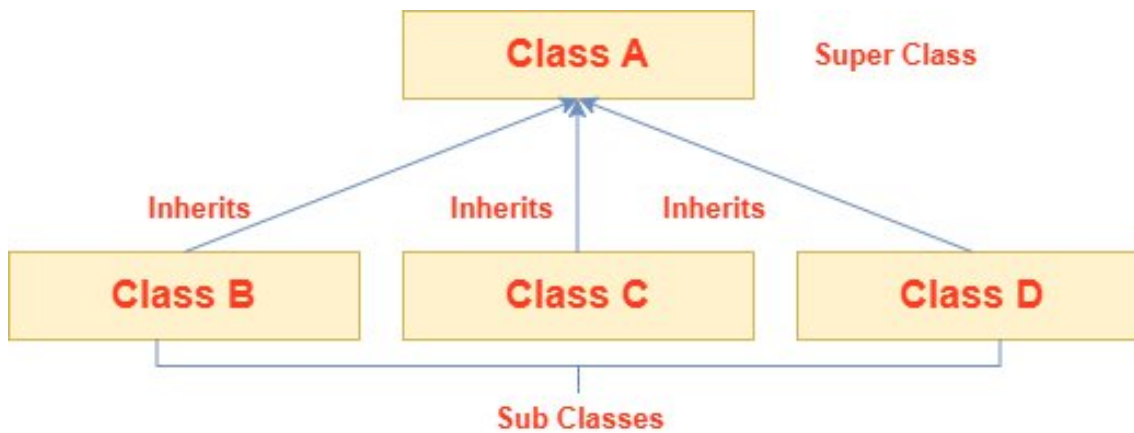
Output:

```
Animal eats  
Dog barks  
Puppy weeps
```

3. Hierarchical Inheritance

Concept : In **hierarchical inheritance**, multiple classes inherit from one superclass.

When two or more classes inherit a single class, it is known as hierarchical inheritance. In the example given below,



Hierarchical Inheritance

Example

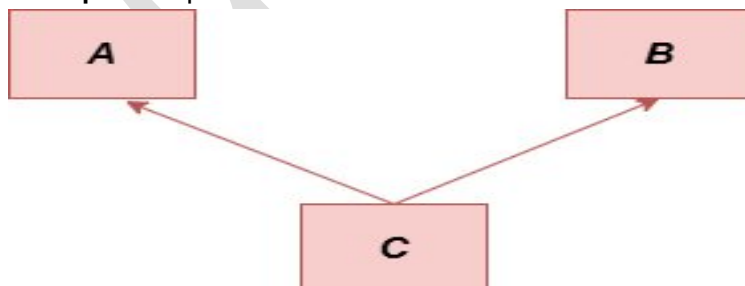
```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}  
class Cat extends Animal {  
    void meow() {  
        System.out.println("Cat meows");  
    }  
}  
Main program  
class Test {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        Cat c = new Cat();  
        d.eat();  
        c.eat();  
    }  
}
```

Output:

Animal eats
Animal eats

4. Multiple Inheritance (Not Supported in Java Classes)

Concept: Multiple inheritance means **one class inherits from multiple parent classes**.



Multiple Inheritance

Example (conceptual):

class C extends A, B

Java **does not allow this with classes.**

Reason: Diamond Problem

Example conflict:

Class A → method show()

Class B → method show()

Class C extends A, B

Java cannot determine which method to use.

How Java Handles Multiple Inheritance

Java supports multiple inheritance using **interfaces**.

Note: interface is inherited with **implements** keyword rather than **extends**.

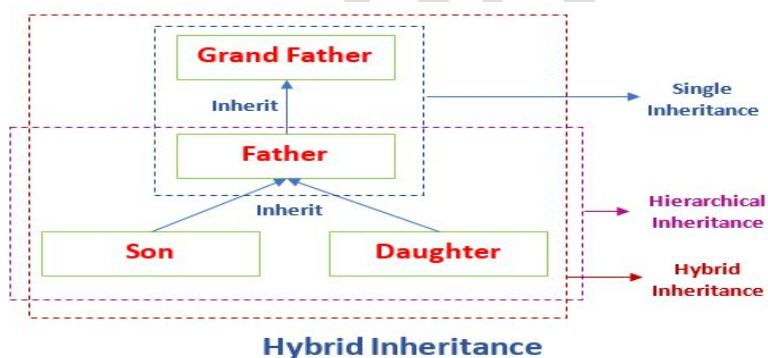
Example:

```
interface A {  
    void show();  
}  
  
interface B {  
    void display();  
}  
  
class C implements A, B {  
    public void show() {  
        System.out.println("Show method");  
    }  
    public void display() {  
        System.out.println("Display method");  
    }  
}
```

5. Hybrid Inheritance

Concept: Hybrid inheritance is a **combination of two or more inheritance types**.

Example:



Java supports hybrid inheritance **using interfaces**.

the hybrid inheritance can be achieved by using the following combinations:

- Single and Multiple Inheritance (not supported but can be achieved through interface)
- Multilevel and Hierarchical Inheritance
- Hierarchical and Single Inheritance
- Multiple and Multilevel Inheritance

Summary Table

Type	Structure	Java Support
Single	One parent → one child	Yes
Multilevel	Parent → child → grandchild	Yes

Hierarchical	One parent → many children	Yes
Multiple	Many parents → one child	Only with interfaces
Hybrid	Combination of types	Using interfaces

Real-World Examples

Inheritance Type	Example
Single	Vehicle → Car
Multilevel	Animal → Dog → Puppy
Hierarchical	Shape → Circle, Rectangle
Multiple	Printer + Scanner → All-in-one device

Unit-4: extends, super, Method Overriding, and Dynamic Method Dispatch (Java)

These concepts explain **how inheritance works internally** and **how runtime polymorphism is achieved in Java**. They are important parts of **Unit-4: Inheritance and Polymorphism**.

1. extends Keyword

Concept: The **extends** keyword is used to inherit one class from another class.

It establishes an **inheritance relationship between two classes**.

Syntax

```
class ChildClass extends ParentClass {
// code
}
```

Here:

Term	Meaning
ParentClass	Superclass
ChildClass	Subclass

Example Program

```
class Animal {
    void eat() {
        System.out.println("Animal eats food");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

Output:

Animal eats food

Dog barks

Dog inherits eat() from Animal.

Key Points

Feature	Explanation
Used for inheritance	Creates parent-child relationship
Subclass inherits methods	Parent methods accessible
Code reuse	Avoid rewriting code

2. super Keyword

Concept: The **super** keyword refers to the immediate parent class object.

It is used to:

1. Access parent class variables
2. Call parent class methods
3. Call parent class constructors

Uses of super

Use	Example
Access parent variable	super.variable
Call parent method	super.method()
Call parent constructor	super()

Example: Access Parent Variable

```
class Animal {  
    String color = "White";  
}  
class Dog extends Animal {  
    String color = "Black";  
    void printColor() {  
        System.out.println(super.color);  
    }  
}  
Main program  
class Test {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.printColor();  
    }  
}
```

Output: White

Example: Calling Parent Method

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
class Dog extends Animal {  
    void eat() {  
        System.out.println("Dog eats");  
    }  
    void display() {  
        super.eat();  
    }  
}
```

```

}
Main program
class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.display();
    }
}

```

Output
Animal eats

Example: Calling Parent Constructor

```

class Animal {
    Animal() {
        System.out.println("Animal constructor");
    }
}
class Dog extends Animal {
    Dog() {
        super();
        System.out.println("Dog constructor");
    }
}

```

```

Main program
class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
    }
}

```

Output
Animal constructor
Dog constructor

Difference Between this and super

Feature	this	super
Refers to	Current class object	Parent class object
Used for	Current class members	Parent class members

1. this and this() in Inheritance

Concept

- **this keyword** refers to the **current class object**
- **this() constructor call** is used to call **another constructor of the same class**

☞ In inheritance, this works within the **current class**, not parent class

Uses of this

Use	Example
Access current variable	this.variable
Call current method	this.method()
Call current constructor	this()

Example 1: Access Current Class Variable

```
class Animal {  
    String color = "White";  
}
```

```
class Dog extends Animal {  
    String color = "Black";  
  
    void printColor() {  
        System.out.println(this.color);  
    }  
}
```

Main Program

```
class Test {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.printColor();  
    }  
}
```

Output

Black

👉 this.color refers to **Dog class variable**, not parent

Example 2: Calling Current Class Method

```
class Dog {  
    void eat() {  
        System.out.println("Dog eats");  
    }  
  
    void display() {  
        this.eat();  
    }  
}
```

Main Program

```
class Test {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.display();  
    }  
}
```

Output

Dog eats

Example 3: Calling Constructor using this()

```
class Dog {  
    Dog() {  
        this(10);  
        System.out.println("Default constructor");  
    }  
  
    Dog(int x) {  
        System.out.println("Parameterized constructor");  
    }  
}
```

```
}  
}
```

Main Program

```
class Test {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
    }  
}
```

Output

Parameterized constructor

Default constructor

👉 `this()` calls another constructor of **same class**

Important Rules of `this()`

- Must be **first statement in constructor**
- Used only for **constructor chaining**
- Cannot be used with `super()` together in same constructor

Key Understanding in Inheritance

- `this` → refers to **current class object (child class)**
- Even in inheritance, `this` never refers to parent class

Difference Between `this` and `super`

Feature	<code>this</code>	<code>super</code>
Refers to	Current class object	Parent class object
Used for	Current class members	Parent class members
Variable access	<code>this.variable</code>	<code>super.variable</code>
Method call	<code>this.method()</code>	<code>super.method()</code>
Constructor call	<code>this()</code>	<code>super()</code>
Usage in inheritance	Works in child class	Access parent class

final Keyword in Java

1. Definition: The final keyword is used to **restrict modification**.

👉 It can be applied to:

- Variables
- Methods
- Classes

2. Uses of final

Use	Meaning
final variable	Value cannot be changed
final method	Cannot be overridden
final class	Cannot be inherited

3. Final Variable

Concept: A final variable becomes a **constant** once initialized.

Example

```
class Test {  
    public static void main(String[] args) {  
        final int x = 10;  
        // x = 20; ✗ Error  
    }  
}
```

```
    System.out.println(x);  
  }  
}
```

Key Points

- Must be initialized once
- Cannot be reassigned
- Used for constants

4. Final Method

Concept: A final method **cannot be overridden** in subclass.

Example

```
class Animal {  
    final void sound() {  
        System.out.println("Animal sound");  
    }  
}
```

```
class Dog extends Animal {  
    // void sound() {} ✗ Error  
}
```

Key Points

- Prevents method overriding
- Ensures original behavior remains unchanged

5. Final Class

Concept: A final class **cannot be extended (inherited)**

Example

```
final class Animal {  
}
```

```
// class Dog extends Animal {} ✗ Error
```

Real Example

- String class in Java is **final**

🔒 Cannot be inherited

6. Final Variable with Objects

```
class Test {  
    public static void main(String[] args) {  
        final StringBuilder sb = new StringBuilder("Hello");  
        sb.append(" Java"); // ✓ Allowed  
        // sb = new StringBuilder("Hi"); ✗ Not allowed  
        System.out.println(sb);  
    }  
}
```

Explanation

- Reference cannot change
- Object content can change

7. Blank Final Variable

Concept: Final variable declared but initialized later

Example

```
class Test {  
    final int x;  
    Test() {  
        x = 10; // must be assigned  
    }  
}
```

8. Static Final Variable

Concept: Used to define constants shared by all objects

Example

```
class Test {  
    static final double PI = 3.14;  
}
```

9. Difference Table

Feature	final variable	final method	final class
Restriction	Cannot change value	Cannot override	Cannot inherit
Applies to	Variables	Methods	Classes

10. Important Rules

- final variable must be initialized once
- Cannot reassign final variable
- Final method cannot be overridden
- Final class cannot be extended
- final ≠ finally ≠ finalize

Method Overloading

Concept: Method overloading occurs when **multiple methods have the same name but different parameters** in the same class.

☞ It is used to **increase readability and flexibility** of code

☞ It is an example of **compile-time polymorphism**

Syntax

```
class ClassName {  
    void methodName() {  
        // code  
    }  
    void methodName(int a) {  
        // overloaded method  
    }  
    void methodName(int a, int b) {  
        // overloaded method  
    }  
}
```

Example Program

```
class Calculator {  
    int add(int a, int b) {
```

```

    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c;
}

double add(double a, double b) {
    return a + b;
}
}

```

Main Program

```

class Test {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(10, 20));
        System.out.println(c.add(10, 20, 30));
        System.out.println(c.add(10.5, 20.5));
    }
}

```

Output

```

30
60
31.0

```

☞ Same method name add() used with different parameters

Rules of Method Overloading

Rule	Explanation
Method name must be same	All methods must have identical name
Parameters must be different	Different number, type, or order
Return type alone is not enough	Cannot overload only by changing return type
Can change data type	Example: int → double
Can change number of arguments	Example: 2 params → 3 params
Can change order of parameters	Example: (int, double) vs (double, int)

Example Showing Rule

```

class Test {
    void show(int a) {
        System.out.println("int method");
    }
    void show(double a) {
        System.out.println("double method");
    }
}

```

Types of Overloading

Type	Example
By number of parameters	add(int, int) vs add(int, int, int)
By data type	add(int, int) vs add(double, double)
By order of parameters	show(int, double) vs show(double, int)

Method Overloading vs Method Overriding

Feature	Overloading	Overriding
Class	Same class	Parent + Child
Method name	Same	Same
Parameters	Different	Same
Return type	Can differ	Must be same/compatible
Polymorphism	Compile-time	Runtime

Compile-Time Polymorphism

Concept: In method overloading, the method to be executed is decided **at compile time** based on parameters.

Example

```
class Test {  
    void show(int a) {  
        System.out.println("int");  
    }  
    void show(String s) {  
        System.out.println("String");  
    }  
    public static void main(String[] args) {  
        Test t = new Test();  
        t.show(10);  
        t.show("Java");  
    }  
}
```

Output

```
int  
String
```

Execution Explanation

- Compiler checks method call
- Matches parameters with method signature
- Calls correct method at compile time

Real-World Example

Calculator System

add(2,3)

add(2,3,4)

add(2.5,3.5)

➤ Same function name, different inputs

Key Points for Exam

Concept	Key Idea
Method overloading	Same method name, different parameters
Polymorphism type	Compile-time
Signature	Method name + parameters
Return type	Not sufficient alone
Use	Improves readability and flexibility

Method Overriding

Concept: Method overriding occurs when a subclass provides its own implementation of a method already defined in the parent class.

It is used to **modify inherited behavior**.

Syntax

```
class Parent {  
    void show() {  
        // code  
    }  
}  
  
class Child extends Parent {  
    void show() {  
        // overridden method  
    }  
}
```

Example Program

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

Main program

```
class Test {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

Output

Dog barks

The child class **overrides the parent method**.

Rules of Method Overriding

Rule	Explanation
Method name must be same	Parent and child
Parameters must be same	Same signature
Return type must match	Or compatible
Cannot reduce access level	Example: public → private not allowed
Static methods cannot be overridden	Only instance methods

Example Showing Rule

```
class A {  
    public void display() {  
        System.out.println("Class A");  
    }  
}
```

```
class B extends A {  
    public void display() {  
        System.out.println("Class B");  
    }  
}
```

Runtime Polymorphism (Method Overriding)

Concept: Runtime polymorphism occurs when a subclass provides a specific implementation of a method already defined in its parent class.

☞ It is used to achieve **dynamic behavior**

☞ It is an example of **runtime polymorphism**

☞ Method call is decided at **runtime (not compile time)**

Syntax

```
class Parent {  
    void methodName() {  
        // code  
    }  
}  
  
class Child extends Parent {  
    void methodName() {  
        // overridden method  
    }  
}
```

Example Program

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat extends Animal {  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}
```

Main Program

```
class Test {  
    public static void main(String[] args) {  
        Animal a;  
        a = new Dog();  
        a.sound();  
        a = new Cat();  
        a.sound();  
    }  
}
```

```
}  
}
```

Output:

Dog barks

Cat meows

☞ Same method sound() behaves differently depending on object

☞ This is **runtime polymorphism**

Dynamic Method Dispatch (Very Important ★)

☞ Mechanism where method call is resolved at runtime

```
Animal a = new Dog();
```

```
a.sound();
```

Type	Value
Reference Type	Animal
Object Type	Dog

☞ Java calls **Dog's method at runtime**

Rules of Method Overriding

Rule	Explanation
Method name must be same	Parent and child
Parameters must be same	Same signature
Return type must be same/compatible	Covariant allowed
Cannot reduce access level	public → private ✗
Static methods cannot be overridden	Only instance methods
Final methods cannot be overridden	Not allowed

Example Showing Rule

```
class A {  
    public void show() {  
        System.out.println("Class A");  
    }  
}
```

```
class B extends A {  
    public void show() {  
        System.out.println("Class B");  
    }  
}
```

Real-Time Example

Payment System

```
class Payment {  
    void pay() {  
        System.out.println("Processing payment");  
    }  
}
```

```
class UPI extends Payment {
```

```

void pay() {
    System.out.println("Paid using UPI");
}
}

```

```

class Card extends Payment {
    void pay() {
        System.out.println("Paid using Card");
    }
}

```

☞ Same method pay() → different implementations

Types of Runtime Polymorphism

Type	Example
Method Overriding	Parent → Child override
Dynamic Method Dispatch	Object decides method

Dynamic Method Dispatch

Concept: Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at runtime rather than compile time.

It is the **basis of runtime polymorphism**.

Example

```

class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

```

Main program:

```

class Test {
    public static void main(String[] args) {
        Animal a;
        a = new Dog();
        a.sound();
    }
}

```

Output

Dog barks

Even though reference type is **Animal**, method of **Dog** executes.

Execution Explanation

Reference Type → Animal

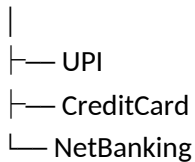
Object Type → Dog

At runtime, Java chooses the **Dog version of the method**.

Real-World Example

Payment System

Payment



Each payment type overrides: processPayment()

Constructor Overloading in Java

1. Definition: Constructor overloading means defining **multiple constructors in the same class with different parameters**.

☞ It allows objects to be initialized in **different ways**

☞ It is a type of **compile-time polymorphism**

2. Syntax

```
class ClassName {
    ClassName() {
        // default constructor
    }
    ClassName(int a) {
        // parameterized constructor
    }

    ClassName(int a, String b) {
        // overloaded constructor
    }
}
```

3. Example Program

```
class Student {
    int id;
    String name;

    // No-argument constructor
    Student() {
        id = 0;
        name = "Unknown";
    }

    // Parameterized constructor
    Student(int i, String n) {
        id = i;
        name = n;
    }

    void display() {
        System.out.println(id + " " + name);
    }

    public static void main(String[] args) {
```

```

Student s1 = new Student();
Student s2 = new Student(101, "Dhruv");

s1.display();
s2.display();
}
}

```

Output

0 Unknown

101 Dhruv

☞ Same class, multiple constructors → **overloading**

4. Rules of Constructor Overloading

Rule	Explanation
Constructor name must be same	Same as class name
Parameters must be different	Number, type, or order
Return type not allowed	Constructors have no return type
Can use this()	For constructor chaining

5. Types of Constructor Overloading

Type	Explanation
No-Argument Constructor	Constructor with no parameters
Parameterized Constructor	Constructor with parameters
Copy Constructor	Constructor that copies another object

6. Type 1: No-Argument Constructor

```

class Test {
    Test() {
        System.out.println("Default constructor");
    }
}

```

7. Type 2: Parameterized Constructor

```

class Test {
    Test(int a) {
        System.out.println("Value: " + a);
    }
}

```

8. Type 3: Copy Constructor

```

class Test {
    int x;

    Test(int x) {
        this.x = x;
    }

    Test(Test t) {
        this.x = t.x;
    }
}

```

9. Constructor Chaining using this()

```

class Test {
    Test() {
        this(10);
        System.out.println("Default");
    }

    Test(int x) {
        System.out.println("Parameterized");
    }
}

```

Output

Parameterized

Default

10. Advantages of Constructor Overloading

- Provides **multiple ways to create objects**
- Improves **code flexibility**
- Reduces need for multiple method names
- Supports **constructor chaining**

11. Important Exam Points

- Constructor overloading = **same name + different parameters**
- Happens in **same class**
- Types:
 - No-argument
 - Parameterized
 - Copy constructor
- `this()` is used for chaining

12. Difference: Method vs Constructor Overloading

Feature	Constructor Overloading	Method Overloading
Name	Same as class	Any name
Purpose	Object initialization	General operation
Return type	Not allowed	Allowed

Polymorphism in Java

Polymorphism is one of the **four fundamental pillars of Object-Oriented Programming (OOP)**.

The word polymorphism means “**many forms**”, meaning **one interface can represent multiple behaviors**.

1. Concept of Polymorphism

Definition: Polymorphism is the ability of a method or object to behave differently depending on the situation.

In Java, the same method name can perform **different operations**.

Meaning of the Word

Poly → Many

Morph → Forms

So polymorphism means:

One method → many behaviors

Real-World Example

Humun { speak() { } }

Different outputs:

English → Hello

Hindi → Namaste

French → Bonjour

Same method **speak()**, different behavior.

Programming Example

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Same method name:

add()

Different behaviors.

Types of Polymorphism in Java

Type	Description
Compile-time polymorphism	Achieved using method overloading
Runtime polymorphism	Achieved using method overriding

2. Compile-Time Polymorphism

Definition: Compile-time polymorphism occurs when **method calls are resolved during compilation**.

It is achieved using **method overloading**.

Method Overloading

Concept: Method overloading means **multiple methods with the same name but different parameters**.

Why It Is Called Compile-Time Polymorphism

The compiler determines **which method to call** during compilation.

3. Runtime Polymorphism

Definition: Runtime polymorphism occurs when **method calls are resolved at runtime instead of compile time.**

It is achieved using:

Method Overriding

Inheritance

Example

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat extends Animal {  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}
```

Main program

```
class Test {  
    public static void main(String[] args) {  
        Animal a;  
        a = new Dog();  
        a.sound();  
        a = new Cat();  
        a.sound();  
    }  
}
```

Output

Dog barks

Cat meows

Same method: sound()

Different behavior depending on object type.

Dynamic Binding

Definition : Dynamic binding means the method call is resolved at runtime based on the object type rather than reference type.

Example

Animal a = new Dog();

a.sound();

Here:

Type	Value
Reference type	Animal
Object type	Dog

Java chooses **Dog's method at runtime.**

Execution Process

Reference Type → Animal

Object Type → Dog

Method chosen:

Dog.sound()

Static Binding vs Dynamic Binding

Feature	Static Binding	Dynamic Binding
Decision time	Compile time	Runtime
Used in	Method overloading	Method overriding
Binding	Early binding	Late binding

Real-World Example

Payment System

Payment

|
└ processPayment()

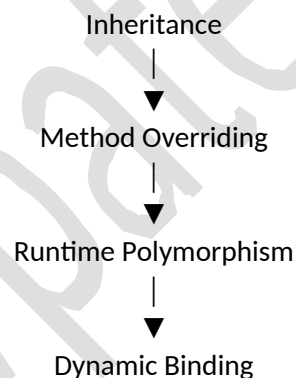
Different implementations:

- UPIPayment
- CardPayment
- NetBankingPayment

Each overrides:

processPayment()

Relationship Between Concepts



Key Points for Exam

Concept	Important Idea
Polymorphism	One interface, many forms
Compile-time polymorphism	Method overloading
Runtime polymorphism	Method overriding
Dynamic binding	Method decided at runtime

Object Casting in Java (Upcasting & Down casting)

1. Definition: Object casting is the process of **converting one object reference type into another**.

☞ Used in **inheritance**

☞ Helps in **runtime polymorphism**

2. Types of Object Casting

Type	Description
Upcasting	Child → Parent
Down casting	Parent → Child

3. Upcasting

Concept: Upcasting means converting a **child class object** into **parent class reference**.

☞ Done **automatically (implicit)**

☞ Always safe

Syntax

```
Parent p = new Child();
```

Example

```
class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

Main Program

```
class Test {  
    public static void main(String[] args) {  
        Animal a = new Dog(); // Upcasting  
        a.sound();  
    }  
}
```

Output

Animal sound

Key Point

- Can access **parent methods only**
- Child-specific methods not accessible

// a.bark(); ✗ Error

4. Down casting

Concept: Down casting means converting **parent reference** back to **child type**

☞ Done **manually (explicit)**

☞ May cause runtime error

Syntax: Child Obj = (Child) parentRef;

Example

```
class Test {  
    public static void main(String[] args) {  
        Animal a = new Dog(); // Upcasting  
        Dog d = (Dog) a;      // Downcasting  
        d.bark();  
    }  
}
```

Output

Dog barks

5. Invalid Downcasting (Runtime Error)

```
Animal a = new Animal();
```

```
Dog d = (Dog) a; // ✗ Runtime Error
```

Error: ClassCastException

☞ Because object is not actually Dog

6. Safe Downcasting (Using instanceof)

```
if (a instanceof Dog) {
```

```
    Dog d = (Dog) a;
```

```
    d.bark();
```

```
}
```

7. Key Differences

Feature	Upcasting	Downcasting
Direction	Child → Parent	Parent → Child
Conversion	Automatic	Manual
Safety	Safe	Risky
Access	Parent members	Child members

8. Memory Understanding

```
Animal a = new Dog();
```

Reference Type → Animal

Object Type → Dog

☞ Actual object decides behavior

9. Real Concept (Important)

- Upcasting enables **dynamic method dispatch**
- Downcasting gives access to **child-specific features**

instanceof Operator in Java

1. Definition: The instanceof operator is used to **check whether an object belongs to a specific class or not**.

☞ It returns a **boolean value**:

- true → if object is instance of class
- false → otherwise

2. Syntax: object instanceof ClassName

3. Basic Example

```
class Test {  
    public static void main(String[] args) {  
        String s = "Java";  
  
        System.out.println(s instanceof String);  
    }  
}
```

Output

true

4. Example with Inheritance

```
class Animal { }
```

```
class Dog extends Animal { }
```

Main Program

```
class Test {  
    public static void main(String[] args) {  
        Animal a = new Dog();  
        System.out.println(a instanceof Animal);  
        System.out.println(a instanceof Dog);  
    }  
}
```

Output

true
true

Explanation

- a instanceof Animal → ✓ true
- a instanceof Dog → ✓ true

☞ Because actual object is **Dog**

5. Important Case (False Condition)

```
Animal a = new Animal();  
System.out.println(a instanceof Dog);  
System.out.println(a instanceof Animal);
```

Output

False
True
☞ Object is not Dog

6. Use in Downcasting (Very Important)

```
if (a instanceof Dog) {  
    Dog d = (Dog) a;  
    d.bark();  
}
```

☞ Prevents **ClassCastException**

7. Special Case: Null

```
Animal a = null;  
System.out.println(a instanceof Animal);
```

Output

false
☞ null is not instance of any class

8. Rules of instanceof

Rule	Explanation
Works with objects only	Not for primitives
Returns boolean	true or false
Checks inheritance	Works with parent-child
Null always returns false	Special case

9. Key Concept (Very Important)

```
Animal a = new Dog();
```

Type	Meaning
Reference type	Animal
Object type	Dog

☞ instanceof checks **object type**, not reference type

11. Comparison with Casting

Feature	instanceof	Casting
Purpose	Check type	Convert type
Safety	Safe	Risky
Result	boolean	Object

Protected Data Members in Java

1. Definition : protected is an **access modifier** in Java.

☞ A **protected data member (variable)** can be accessed:

- Within the **same class**
- Within the **same package**
- In **subclasses (child classes)** even in different packages

2. Syntax

protected int value;

3. Access Scope of protected

Location	Accessible
Same class	✓ Yes
Same package (non-subclass)	✓ Yes
Subclass (same package)	✓ Yes
Subclass (different package)	✓ Yes
Non-subclass (different package)	✗ No

4. Example (Same Package)

```
class Animal {  
    protected String type = "Animal";  
}  
  
class Dog extends Animal {  
    void display() {  
        System.out.println(type);  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.display();  
    }  
}
```

Output

Animal

5. Example (Different Package with Inheritance)

Package: pack1

```
package pack1;  
  
public class Animal {  
    protected String type = "Animal";  
}
```

Package: pack2

```
package pack2;
import pack1.Animal;
class Dog extends Animal {
    void show() {
        System.out.println(type); // ✓ Accessible
    }
}
```

6. Important Rule (Very Important)

☞ In different package:

```
Dog d = new Dog();
System.out.println(d.type); // ✗ Error
```

! Even though it's protected, it **cannot be accessed using object reference outside package**

✓ Only accessible through **inheritance**

7. Difference Between Access Modifiers

Modifier	Same Class	Same Package	Subclass (diff pkg)	Other
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

String in Java

1. Definition: A **String** in Java is a **sequence of characters** used to represent text.

☞ In Java, String is an **object**, not a primitive data type

☞ It belongs to java.lang package

2. Ways to Create String

1. Using String Literal

```
String s1 = "Hello";
```

☞ Stored in **String Constant Pool (SCP)**

☞ Memory efficient

2. Using new Keyword

```
String s2 = new String("Hello");
```

☞ Stored in **Heap memory**

☞ Creates new object every time

3. String is Immutable (Very Important)

Definition : Once a String object is created, **its value cannot be changed**

Example

```
String s = "Java";
s = s.concat(" Programming");
System.out.println(s);
```

Output

Java Programming

☞ A **new object is created**, original remains unchanged

4. String Memory Concept

```
String s1 = "Java";
```

```
String s2 = "Java";
```

☞ Both refer to **same memory in SCP**

```
String s3 = new String("Java");
```

☞ New object created in heap

5. Common String Methods

1. length()

```
String s = "Java";
```

```
System.out.println(s.length());
```

☞ Output: 4

2. charAt()

```
System.out.println(s.charAt(1));
```

☞ Output: a

3. toUpperCase() / toLowerCase()

```
s.toUpperCase(); // JAVA
```

```
s.toLowerCase(); // java
```

4. equals()

```
s1.equals(s2);
```

☞ Compares **content**

5. equalsIgnoreCase()

```
"java".equalsIgnoreCase("JAVA");
```

6. compareTo()

```
"abc".compareTo("bcd");
```

☞ Used for **lexicographic comparison**

7. concat()

```
s1.concat(s2);
```

8. substring()

```
String s = "JavaProgramming";
```

```
System.out.println(s.substring(4));
```

☞ Output: Programming

9. contains()

```
s.contains("Java");
```

10. replace()

```
s.replace('a', 'x');
```

6. String Comparison

Using ==

```
String s1 = "Java";
```

```
String s2 = "Java";
```

```
System.out.println(s1 == s2);
```

☞ true (same reference)

Using .equals()

```
String s1 = new String("Java");
String s2 = new String("Java");
System.out.println(s1.equals(s2));
```

👉 true (same content)

7. Mutable String Classes

Class	Description
String	Immutable
StringBuffer	Mutable, thread-safe
StringBuilder	Mutable, faster

Example

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" Programming");
System.out.println(sb);
```

8. Difference Table

Feature	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Thread-safe	Yes	Yes	No
Performance	Slow	Medium	Fast

9. Important Concepts

String Constant Pool (SCP)

- Special memory area in heap
- Stores string literals
- Avoids duplicate objects

Intern Method

```
String s1 = new String("Java");
String s2 = s1.intern();
```

👉 Moves string to SCP

10. Important Exam Points

- String is **immutable**
- Stored in **heap + SCP**
- **==** compares reference
- **.equals()** compares content
- **StringBuffer & StringBuilder** are mutable

StringBuffer in Java

1. Definition: **StringBuffer** is a **mutable class** used to create and modify strings dynamically.

👉 Unlike String, **StringBuffer can change its content without creating new objects**

👉 It belongs to java.lang package

2. Key Feature

- ✓ **Mutable (changeable)**
- ✓ **Thread-safe (synchronized)**

3. Why StringBuffer is Used

- When frequent **modification of strings** is required
- To improve **performance** (avoids creating multiple objects)

- In multi-threaded environments

4. Creating StringBuffer

```
StringBuffer sb1 = new StringBuffer();    // empty  
StringBuffer sb2 = new StringBuffer("Java");    // with value
```

5. Common Methods of StringBuffer

1. append()

```
StringBuffer sb = new StringBuffer("Java");  
sb.append(" Programming");  
System.out.println(sb);
```

➤ Output: Java Programming

2. insert()

```
sb.insert(4, " Language");
```

➤ Inserts text at given index

3. replace()

```
sb.replace(0, 4, "C++");
```

➤ Replaces characters from index range

4. delete()

```
sb.delete(0, 4);
```

➤ Deletes characters from index range

5. reverse()

```
sb.reverse();
```

➤ Reverses string

6. capacity()

```
System.out.println(sb.capacity());
```

➤ Returns current capacity

7. length()

```
System.out.println(sb.length());
```

➤ Returns number of characters

8. charAt()

```
System.out.println(sb.charAt(2));
```

9. setCharAt()

```
sb.setCharAt(0, 'M');
```

➤ Changes character at index

6. Example Program

```
class Test {  
    public static void main(String[] args) {  
        StringBuffer sb = new StringBuffer("Java");  
  
        sb.append(" Programming");  
        sb.insert(4, " Language");
```

```
        System.out.println(sb);
    }
}
```

Output

Java Language Programming

7. Capacity Concept (Very Important)

Default Capacity: 16 + length of string

Example:

```
StringBuffer sb = new StringBuffer("Java");
System.out.println(sb.capacity());
```

🔗 Output: 20

8. String vs StringBuffer

Feature	String	StringBuffer
Mutability	Immutable	Mutable
Performance	Slow (new object)	Fast (same object)
Thread-safe	Yes	Yes
Memory	More usage	Less usage

9. StringBuffer vs StringBuilder

Feature	StringBuffer	StringBuilder
Thread-safe	Yes	No
Speed	Slower	Faster
Use case	Multi-threaded	Single-threaded

10. Important Exam Points

- StringBuffer is **mutable**
- It is **synchronized (thread-safe)**
- Methods:
 - append()
 - insert()
 - replace()
 - delete()
 - reverse()
- Default capacity = **16 + initial size**

StringBuilder in Java

1. Definition: **StringBuilder** is a **mutable class** used to create and modify strings.

🔗 Similar to StringBuffer

🔗 But **NOT thread-safe** (no synchronization)

2. Key Features

- ✓ Mutable (can change content)
- ✓ Faster than StringBuffer
- ✗ Not thread-safe

3. Why StringBuilder is Used

- When frequent string modification is needed
- When performance is important
- Used in **single-threaded applications**

4. Creating StringBuilder

```
StringBuilder sb1 = new StringBuilder();    // empty
StringBuilder sb2 = new StringBuilder("Java"); // with value
```

5. Common Methods of StringBuilder

1. append()

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" Programming");
System.out.println(sb);
```

🔗 Output: Java Programming

2. insert()

```
sb.insert(4, " Language");
```

🔗 Inserts text at given index

3. replace()

```
sb.replace(0, 4, "C++");
```

4. delete()

```
sb.delete(0, 4);
```

5. reverse()

```
sb.reverse();
```

6. capacity()

```
System.out.println(sb.capacity());
```

7. length()

```
System.out.println(sb.length());
```

8. charAt()

```
System.out.println(sb.charAt(2));
```

9. setCharAt()

```
sb.setCharAt(0, 'M');
```

6. Example Program

```
class Test {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Java");

        sb.append(" Programming");
        sb.insert(4, " Language");

        System.out.println(sb);
    }
}
```

Output

Java Language Programming

7. Capacity

Default Capacity: 16 + length of string

Example:

```
StringBuilder sb = new StringBuilder("Java");  
System.out.println(sb.capacity());
```

🔗 Output: 20

8. String vs StringBuilder

Feature	String	StringBuilder	StringBuffer	StringBuffer
Mutability	Immutable	Mutable	✗ No	✓ Yes
Performance	Slow	Fast	Faster	Slower
Thread-safe	Yes	No	Single-threaded	Multi-threaded

10. Important Exam Points

- StringBuilder is **mutable and fast**
- Not synchronized → **not thread-safe**
- Same methods as StringBuffer
- Default capacity = **16 + length**

Wrapper Classes in Java

Wrapper classes are used to **convert primitive data types into objects**.

They belong to the java.lang package and are important when working with **collections, object manipulation, and utility methods**.

1. Wrapper Class Concept

Definition: A **wrapper class** is a class that **wraps (encapsulates)** a **primitive data type** into an **object**.

Primitive data types are not objects, but many Java APIs require **objects instead of primitives**.

Wrapper classes solve this problem.

Primitive vs Wrapper

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Example

Primitive variable:

```
int number = 10;
```

Wrapper object:

```
Integer obj = Integer.valueOf(10);
```

Here:

int → primitive

Integer → object

Why Wrapper Classes Are Needed

Reason	Explanation
Collections require objects	Example: ArrayList cannot store primitives
Utility methods	Wrapper classes provide useful methods
Object manipulation	Objects can be passed easily

2. Primitive → Object Conversion (Autoboxing)

Concept: Converting a **primitive data type** into a **wrapper object** automatically.

This is called **Autoboxing**.

Example

```
int num = 10;
```

```
Integer obj = num;
```

Java automatically converts:

```
int → Integer
```

Example Program

```
class Test {  
    public static void main(String[] args) {  
        int num = 20;  
        Integer obj = num; // Autoboxing  
        System.out.println(obj);  
    }  
}
```

Output

20

Manual Conversion

Before Java 5, conversion was done manually.

```
Integer obj = Integer.valueOf(10);
```

3. Object → Primitive Conversion (Unboxing)

Concept: Converting a **wrapper object** back into a **primitive type**.

This process is called **Unboxing**.

Example

```
Integer obj = 30;
```

```
int num = obj;
```

Here:

```
Integer → int
```

Java automatically extracts the primitive value.

Example Program

```
class Test {  
    public static void main(String[] args) {  
        Integer obj = 50;  
        int num = obj;  
        System.out.println(num);  
    }  
}
```

Output:

50

Manual Unboxing

```
Integer obj = Integer.valueOf(40);
```

```
int num = obj.intValue();
```

Wrapper Class Methods

Wrapper classes provide useful methods.

Example:

```
Integer.parseInt("123");
```

```
Double.parseDouble("10.5");
```

Example Program Using Wrapper Classes

```
class Test {  
    public static void main(String[] args) {  
        String str = "100";  
        int number = Integer.parseInt(str);  
        System.out.println(number + 50);  
    }  
}
```

Output

150

Memory Representation

Primitive:

Stack

num = 10

Wrapper Object:

Stack

obj → reference

Heap

Integer Object (10)

Real-World Example

Student Marks Processing

Input from user:

"85"

Convert string to integer:

```
int marks = Integer.parseInt("85");
```

Now arithmetic operations can be performed.

Key Differences

Feature	Primitive Type	Wrapper Class
Data type	Basic data	Object representation
Memory	Stored directly	Stored as object
Methods	No methods	Many utility methods

Java API Packages

1. Definition: Java API (Application Programming Interface) packages are **predefined libraries** provided by Java to perform common tasks.

🔗 They contain:

- Classes
- Interfaces
- Methods

🔗 Stored in **Java Standard Library**

2. Why Java API Packages are Used

- Reduce coding effort
- Provide ready-made functionality
- Improve development speed
- Ensure reliability

3. Common Java API Packages

Package	Description
java.lang	Core classes (String, Math, System)
java.util	Utility classes (Scanner, ArrayList)
java.io	Input/output operations
java.net	Networking
java.sql	Database connectivity
java.awt	GUI components
javax.swing	Advanced GUI
java.time	Date and time handling

4. Important Packages in Detail

1. java.lang (Most Important)

Features

- Automatically imported
- Contains fundamental classes

Common Classes

- String
- Math
- System
- Object

Example

```
System.out.println("Hello");
```

2. java.util

Features

- Utility classes and data structures

Common Classes

- Scanner
- ArrayList
- HashMap
- Date

Example

```
import java.util.Scanner;
```

```
Scanner sc = new Scanner(System.in);
```

3. java.io

Features

- Input and output operations

Common Classes

- File
- BufferedReader
- InputStream

- OutputStream

Example

```
import java.io.*;
```

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
```

4. java.net

Features

- Networking operations

Common Classes

- URL
- Socket
- ServerSocket

5. java.sql

Features

- Database connectivity (JDBC)

Common Classes

- Connection
- Statement
- ResultSet

6. java.awt and javax.swing

Features

- GUI development

Package	Use
java.awt	Basic GUI
javax.swing	Advanced GUI

7. java.time

Features

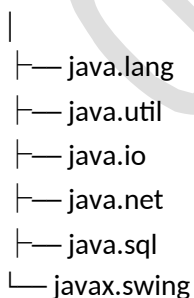
- Modern date and time API

Common Classes

- LocalDate
- LocalTime
- LocalDateTime

Structure of Java API

Java API



1. Autoboxing and Unboxing in Java

Concept: Java provides automatic conversion between **primitive types** and **wrapper classes**.

Type	Meaning
------	---------

Autoboxing	Primitive → Wrapper object
Unboxing	Wrapper object → Primitive

1. Autoboxing

Definition : Automatic conversion of **primitive into object**

Example

```
int a = 10;
```

```
Integer obj = a; // Autoboxing
```

☞ Internally:

```
Integer obj = Integer.valueOf(a);
```

2. Unboxing

Definition: Automatic conversion of **object into primitive**

Example

```
Integer obj = 20;
```

```
int a = obj; // Unboxing
```

☞ Internally:

```
int a = obj.intValue();
```

3. Example Program

```
class Test {
    public static void main(String[] args) {
        int a = 10;

        // Autoboxing
        Integer obj = a;

        // Unboxing
        int b = obj;

        System.out.println(a + " " + obj + " " + b);
    }
}
```

4. Why Autoboxing is Needed

- Collections store **objects only**
- Simplifies code (no manual conversion)

5. Important Points

- Introduced in **Java 5**
- Works with all wrapper classes
- Reduces boilerplate code

Quick Recall

- Autoboxing → primitive → object
- Unboxing → object → primitive

2. Collection Framework in Java

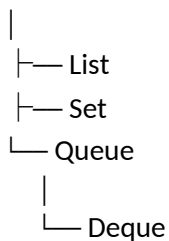
1. Definition: Java Collection Framework is a **set of classes and interfaces** used to store and manipulate groups of objects.

2. Why Collection Framework

- Dynamic size (unlike arrays)
- Ready-made data structures
- Easy data manipulation

3. Hierarchy Overview

Collection



4. Main Interfaces

Interface	Description
Collection	Root interface
List	Ordered, allows duplicates
Set	Unordered, no duplicates
Queue	FIFO structure
Map	Key-value pairs (separate hierarchy)

5. List Interface

Features

- Ordered
- Allows duplicates

Classes

- ArrayList
- LinkedList
- Vector

Example

```
import java.util.*;
```

```
ArrayList<Integer> list = new ArrayList<>();  
list.add(10);  
list.add(20);
```

6. Set Interface

Features

- No duplicates
- Unordered

Classes

- HashSet
- LinkedHashSet
- TreeSet

7. Queue Interface

Features

- FIFO (First In First Out)

Classes

- PriorityQueue

- ArrayDeque

8. Map Interface

Features

- Key-value pairs
- No duplicate keys

Classes

- HashMap
- TreeMap
- LinkedHashMap

9. Important Methods

Method	Use
add()	Add element
remove()	Remove element
size()	Number of elements
contains()	Check element
clear()	Remove all

10. Example Program

```
import java.util.*;
```

```
class Test {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();

        list.add("Java");
        list.add("Python");

        System.out.println(list);
    }
}
```

11. Advantages

- Dynamic size
- Built-in algorithms
- Efficient data handling

12. Collection vs Array

Feature	Array	Collection
Size	Fixed	Dynamic
Type	Primitive/Object	Object only
Flexibility	Low	High

13. Important Exam Points

- Collection stores **objects only**
- List → duplicates allowed
- Set → no duplicates
- Map → key-value pairs
- Most used: **ArrayList, HashSet, HashMap**

User-Defined Packages in Java

1. Definition: A **user-defined package** is a package created by the programmer to **organize related classes and interfaces**.

☞ It is used to:

- Group related classes
- Avoid name conflicts
- Improve code structure and reusability

2. Why User-Defined Packages are Needed

- To manage **large projects**
- To avoid **class name duplication**
- To provide **access control**
- To improve **modularity**

3. Creating a User-Defined Package

Step 1: Declare Package

```
package mypackage;
```

☞ Must be written at the **top of the program**

Step 2: Create Class

```
package mypackage;
public class Hello {
    public void display() {
        System.out.println("Hello from package");
    }
}
```

Step 3: Compile Program

```
javac -d . Hello.java
```

☞ `-d .` creates directory structure automatically

Folder Structure Created

```
mypackage/
├── Hello.class
```

4. Using User-Defined Package

Method 1: Import Package

```
import mypackage.Hello;
class Test {
    public static void main(String[] args) {
        Hello h = new Hello();
        h.display();
    }
}
```

Method 2: Fully Qualified Name

```
class Test {
    public static void main(String[] args) {
        mypackage.Hello h = new mypackage.Hello();
        h.display();
    }
}
```

}

5. Types of Packages

Type	Description
Built-in packages	Provided by Java (java.util, java.io)
User-defined packages	Created by programmer

6. Access Rules in Packages

Modifier	Same Package	Different Package
private	✗	✗
default	✓	✗
protected	✓	✓ (via inheritance)
public	✓	✓

7. Subpackages

Definition: A package inside another package

Example

```
package com.company.project;
```

☞ Creates hierarchy:

```
com/  
└─ company/  
    └─ project/
```

8. Example (Complete Program)

Step 1: Package Class

```
package mypack;  
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

Step 2: Main Class

```
import mypack.Calculator;  
class Test {  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
        System.out.println(c.add(10, 20));  
    }  
}
```

Output

30

9. Important Commands

Command	Use
javac -d . filename.java	Compile and create package folder

<code>java</code> <code>packageName.ClassName</code>	Run program
---	-------------

10. Advantages of User-Defined Packages

- Avoids **name conflicts**
- Improves **code organization**
- Supports **access control**
- Enables **reuse of code**
- Makes large projects manageable

12. Important Rules

- Package statement must be **first line**
- Directory structure must match package name
- Class must be **public** to access outside package
- Use import to access classes

13. Real-World Example

`com.company.project.module`

🔑 Used in large applications like:

- Banking systems
- E-commerce apps